Register Number: 192425317

Name: M. Sahasra

CO3 – AT1

MLA0201 – Fundamentals of Machine Learning

Experiments

1) Customer Segmentation using K-Means and PCA: Load the Mall Customers dataset and preprocess the data. Apply the K-Means algorithm to identify customer clusters, then use PCA to reduce the data to two dimensions and visualize the clusters. Analyze the optimal number of clusters using the Elbow Method and Silhouette Score.

Answer)

Definitions:

Unsupervised Learning:

Unsupervised learning is a machine learning technique in which the algorithm learns patterns from unlabelled data. There is no predefined target/output variable.

Clustering:

Clustering is an unsupervised learning technique that divides data into groups called clusters, where similar data points are placed together.

K-Means Clustering:

K-Means is a clustering algorithm that divides data into K clusters.

The basic steps are:

1. Select the number of clusters, K.
2. Initialize K centroids.

3. Assign each data point to its nearest centroid.
4. Recalculate the centroids.
5. Repeat until the clusters stabilize.

Centroid:

A centroid is the central point representing a cluster. In K-Means, the centroid is calculated as the mean of all points belonging to that cluster.

PCA — Principal Component Analysis:

PCA is a dimensionality reduction technique that transforms a large number of correlated features into a smaller number of principal components while preserving as much information as possible.

Elbow Method:

The Elbow Method helps determine a suitable value of K by plotting Within-Cluster Sum of Squares (WCSS) against different values of K.

The point where the decrease in WCSS becomes slower is called the elbow point.

Silhouette Score:

Silhouette Score measures how well each data point fits within its assigned cluster.

Its value ranges from:

-1 to +1

A value closer to +1 generally indicates better-defined clusters.

Objective:

To segment mall customers into different groups using K-Means clustering, determine the optimal number of clusters using the Elbow Method and Silhouette Score, and visualize the clusters using PCA.

Python Code:

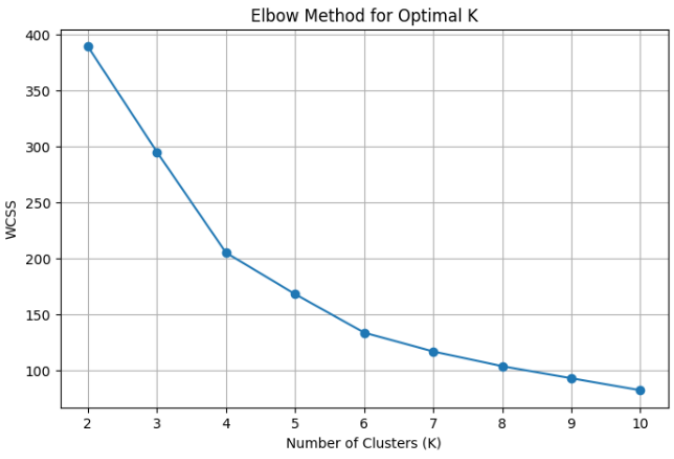
```python
# ============================================================
# EXPERIMENT 1
# Customer Segmentation using K-Means and PCA
# ============================================================

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

# ------------------------------------------------------------
# STEP 1: Load Mall Customers Dataset
# ------------------------------------------------------------

url = "https://raw.githubusercontent.com/sharmaroshan/Clustering-of-Mall-Customers/master/Mall_Customers.csv"

df = pd.read_csv(url)

print("First 5 rows of the dataset:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

# ------------------------------------------------------------
# STEP 2: Data Preprocessing
# ------------------------------------------------------------

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Select numerical features
features = [
    "Age",
    "Annual Income (k$)",
    "Spending Score (1-100)"
]

X = df[features]

# Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\nStandardized data (first 5 rows):")
print(X_scaled[:5])

# ------------------------------------------------------------
# STEP 3: Elbow Method
# ------------------------------------------------------------

wcss = []

for k in range(2, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(
    range(2, 11),
    wcss,
    marker='o'
)

plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.grid(True)
plt.show()
```

```python

# ------------------------------------------------------------
# STEP 4: Silhouette Score
# ------------------------------------------------------------

silhouette_scores = []

for k in range(2, 11):

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = kmeans.fit_predict(X_scaled)

    score = silhouette_score(X_scaled, labels)

    silhouette_scores.append(score)

# Display scores
print("\nSilhouette Scores:")

for k, score in zip(range(2, 11), silhouette_scores):
    print(f"K = {k}, Silhouette Score = {score:.4f}")

# Find best K according to silhouette score
best_k = range(2, 11)[np.argmax(silhouette_scores)]

print("\nBest K according to Silhouette Score:", best_k)

# Plot silhouette scores
plt.figure(figsize=(8, 5))

plt.plot(
    range(2, 11),
    silhouette_scores,
    marker='o'
)

plt.title("Silhouette Score for Different K Values")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.grid(True)
plt.show()

# ------------------------------------------------------------
# STEP 5: Apply K-Means using Best K
# ------------------------------------------------------------

kmeans = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(X_scaled)

# Add cluster labels to original dataframe
df["Cluster"] = clusters

print("\nCustomer Cluster Assignment:")
print(df.head(10))

# ------------------------------------------------------------
# STEP 6: Apply PCA
# ------------------------------------------------------------

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

print("\nPCA Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

print(
    "\nTotal Variance Explained:",
    round(sum(pca.explained_variance_ratio_) * 100, 2),
    "%"
)

```

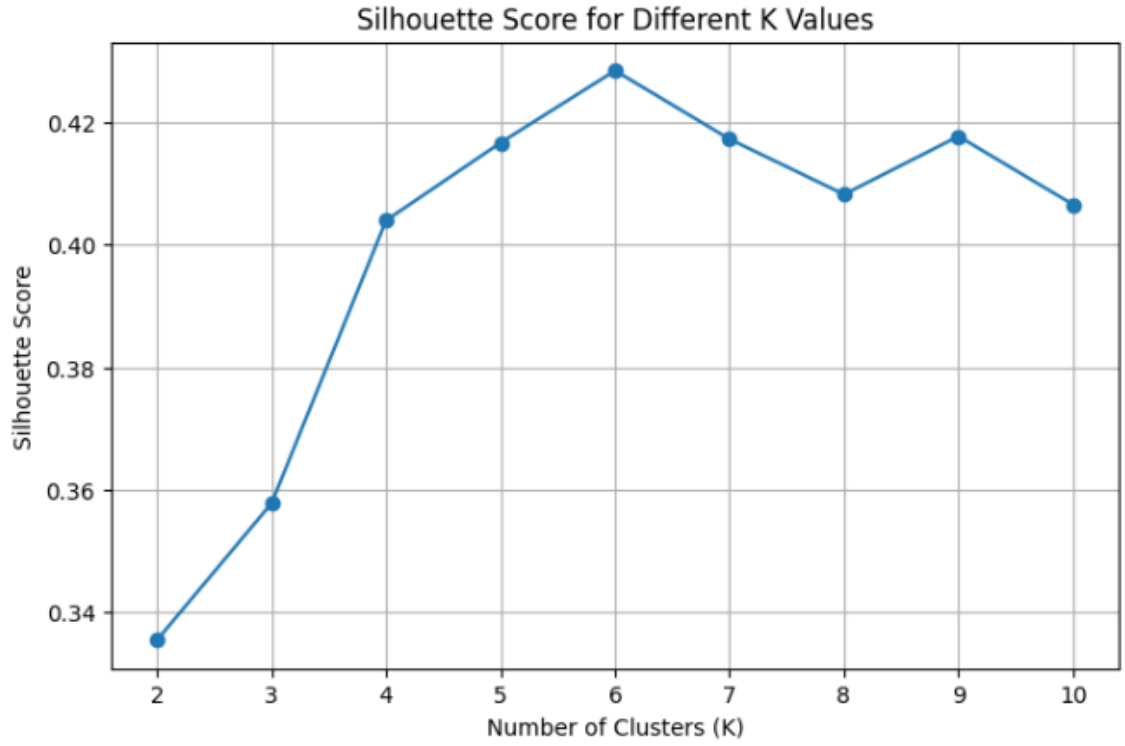
```
# ------------------------------------------------------------
# STEP 7: Visualize Clusters using PCA
# ------------------------------------------------------------

plt.figure(figsize=(9, 6))

scatter = plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=clusters,
    cmap="viridis",
    s=50
)

plt.title("Customer Segmentation using K-Means + PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.colorbar(scatter, label="Cluster")

plt.grid(True)
plt.show()

# ------------------------------------------------------------
# STEP 8: Cluster Summary
# ------------------------------------------------------------

print("\nCluster Summary:")

summary = df.groupby("Cluster")[features].mean()

print(summary)

# ------------------------------------------------------------
# FINAL RESULT
# ------------------------------------------------------------

print("\n====================================")
print("EXPERIMENT 1 COMPLETED")
print("====================================")
print("Optimal K:", best_k)
print("Number of Customers:", len(df))
print(
    "PCA Variance Explained:",
    round(sum(pca.explained_variance_ratio_) * 100, 2),
    "%"
)
```

Output:

```
First 5 rows of the dataset:
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Dataset Shape:
(200, 5)

Missing Values:
CustomerID              0
Genre                   0
Age                     0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64

Standardized data (first 5 rows):
[[-1.42456879 -1.73899919 -0.43480148]
 [-1.28103541 -1.73899919  1.19570407]
 [-1.3528021  -1.70082976 -1.71591298]
 [-1.13750203 -1.70082976  1.04041783]
 [-0.56336851 -1.66266033 -0.39597992]]
```



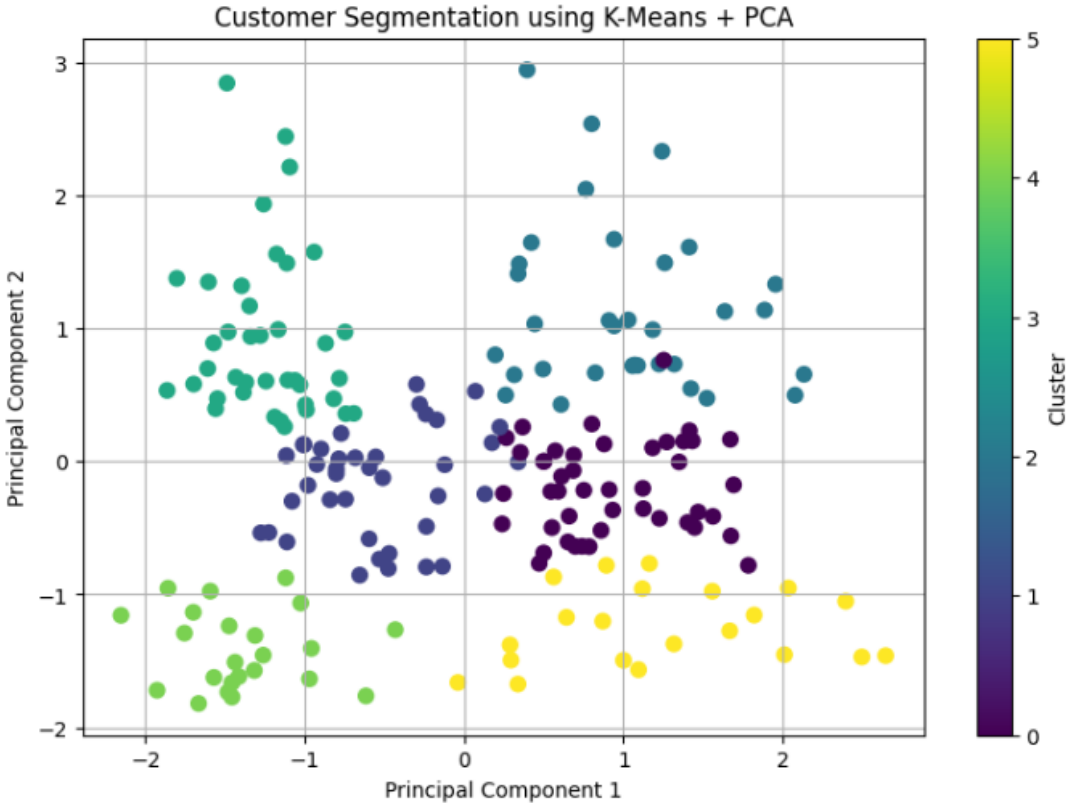
```
Silhouette Scores:
K = 2, Silhouette Score = 0.3355
K = 3, Silhouette Score = 0.3578
K = 4, Silhouette Score = 0.4040
K = 5, Silhouette Score = 0.4166
K = 6, Silhouette Score = 0.4284
K = 7, Silhouette Score = 0.4172
K = 8, Silhouette Score = 0.4082
K = 9, Silhouette Score = 0.4177
K = 10, Silhouette Score = 0.4066

Best K according to Silhouette Score: 6
```



Customer Cluster Assignment:

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)	\
0	1	Male	19	15	39	
1	2	Male	21	15	81	
2	3	Female	20	16	6	
3	4	Female	23	16	77	
4	5	Female	31	17	40	
5	6	Female	22	17	76	
6	7	Female	35	18	6	
7	8	Female	23	18	94	
8	9	Male	64	19	3	
9	10	Female	30	19	72	

```
   Cluster
0        4
1        4
2        5
3        4
4        5
5        4
6        5
7        4
8        5
9        4

PCA Explained Variance Ratio:
[0.44266167 0.33308378]

Total Variance Explained: 77.57 %
```




```
Cluster Summary:
                Age  Annual Income (k$)  Spending Score (1-100)
Cluster
0         56.333333           54.266667               49.066667
1         26.794872           57.102564               48.128205
2         41.939394           88.939394               16.969697
3         32.692308           86.538462               82.128205
4         25.000000           25.260870               77.608696
5         45.523810           26.285714               19.380952

========================================
EXPERIMENT 1 COMPLETED
========================================
Optimal K: 6
Number of Customers: 200
PCA Variance Explained: 77.57 %
```

Result:

Thus, K-Means clustering successfully segmented the mall customers into groups, and PCA was used to represent the clusters in two dimensions. The Elbow Method and Silhouette Score were used to determine an appropriate number of clusters.

2) Clustering using EM Algorithm and Gaussian Mixture Model: Load the Digits dataset from Scikit-learn and preprocess the features. Implement the Gaussian Mixture Model (EM Algorithm) to cluster the data and compare its performance with K-Means using appropriate clustering evaluation metrics. Visualize the clusters after applying PCA.

Answer)

Definitions:

Gaussian Mixture Model — GMM:

A Gaussian Mixture Model is a probabilistic clustering model that assumes the data is generated from a mixture of several Gaussian probability distributions.

Each Gaussian distribution represents a cluster.

EM Algorithm:

EM stands for Expectation-Maximization.

It is an iterative optimization algorithm used to estimate parameters when some information is hidden or unknown.

It has two main steps:

E-Step — Expectation:

Estimate the probability that each data point belongs to each cluster.

M-Step — Maximization:

Update the model parameters based on those probabilities.

These steps are repeated until the model converges.

Hard Clustering:

In hard clustering, each data point belongs to exactly one cluster.

K-Means is an example of hard clustering.

Soft Clustering:

In soft clustering, a data point can belong to multiple clusters with different probabilities.

GMM performs soft clustering.

K-Means vs GMM:

K-Means	**GMM**
Distance-based	Probability-based
Hard clustering	Soft clustering
Uses centroids	Uses Gaussian distributions
Assigns one cluster	Provides cluster probabilities
Simpler	More flexible

Objective:

To cluster handwritten digit data using Gaussian Mixture Model with the EM algorithm, compare its performance with K-Means, and visualize the resulting clusters using PCA.

Python Code:

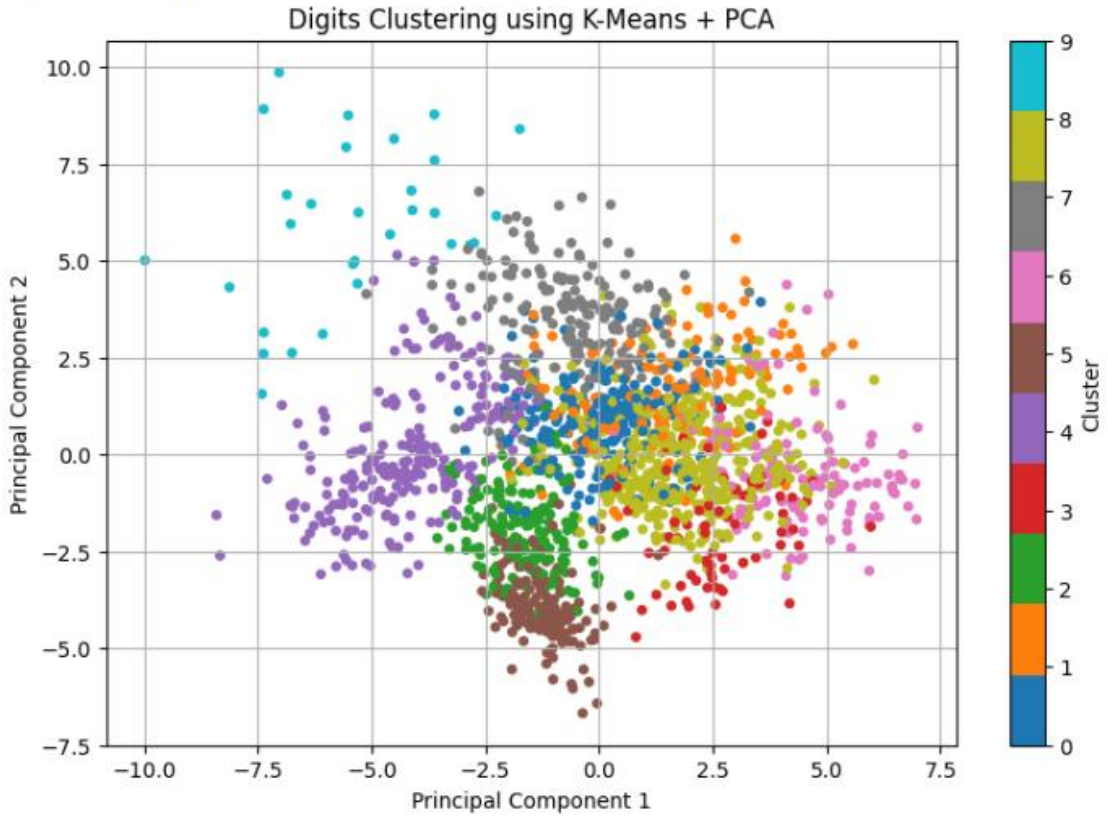
```python
# ============================================================
# EXPERIMENT 2
# Clustering using EM Algorithm and Gaussian Mixture Model
# ============================================================

# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.decomposition import PCA
from sklearn.metrics import (
    silhouette_score,
    adjusted_rand_score,
    normalized_mutual_info_score
)

# ------------------------------------------------------------
# STEP 1: Load Digits Dataset
# ------------------------------------------------------------

digits = load_digits()

X = digits.data
y = digits.target

print("Dataset Shape:")
print(X.shape)

print("\nNumber of Classes:")
print(len(np.unique(y)))

print("\nFirst 5 target values:")
print(y[:5])

# ------------------------------------------------------------
# STEP 2: Standardize Features
# ------------------------------------------------------------

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\nStandardized data shape:")
print(X_scaled.shape)

# ------------------------------------------------------------
# STEP 3: Apply K-Means
# ------------------------------------------------------------

# Digits dataset contains 10 digit classes
n_clusters = 10

kmeans = KMeans(
    n_clusters=n_clusters,
    random_state=42,
    n_init=10
)

kmeans_labels = kmeans.fit_predict(X_scaled)

# ------------------------------------------------------------
# STEP 4: Apply Gaussian Mixture Model
# ------------------------------------------------------------

gmm = GaussianMixture(
    n_components=n_clusters,
    covariance_type="full",
    random_state=42
)

gmm_labels = gmm.fit_predict(X_scaled)

# ------------------------------------------------------------
# STEP 5: Evaluate K-Means
# ------------------------------------------------------------

kmeans_silhouette = silhouette_score(
```

```
    X_scaled,
    kmeans_labels
)

kmeans_ari = adjusted_rand_score(
    y,
    kmeans_labels
)

kmeans_nmi = normalized_mutual_info_score(
    y,
    kmeans_labels
)

# ------------------------------------------------------------
# STEP 6: Evaluate GMM
# ------------------------------------------------------------

gmm_silhouette = silhouette_score(
    X_scaled,
    gmm_labels
)

gmm_ari = adjusted_rand_score(
    y,
    gmm_labels
)

gmm_nmi = normalized_mutual_info_score(
    y,
    gmm_labels
)

# ------------------------------------------------------------
# STEP 7: Display Comparison
# ------------------------------------------------------------

results = pd.DataFrame({
    "Algorithm": [
        "K-Means",
        "Gaussian Mixture Model"
    ],

    "Silhouette Score": [
        kmeans_silhouette,
        gmm_silhouette
    ],

    "Adjusted Rand Index": [
        kmeans_ari,
        gmm_ari
    ],

    "Normalized Mutual Information": [
        kmeans_nmi,
        gmm_nmi
    ]
})

print("\n================================================")
print("CLUSTERING PERFORMANCE COMPARISON")
print("================================================")

print(results.to_string(index=False))

# ------------------------------------------------------------
# STEP 8: PCA for Visualization
# ------------------------------------------------------------

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

print("\nPCA Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

print(
    "Total Variance Explained:",
    round(sum(pca.explained_variance_ratio_) * 100, 2),
    "%"
)
```

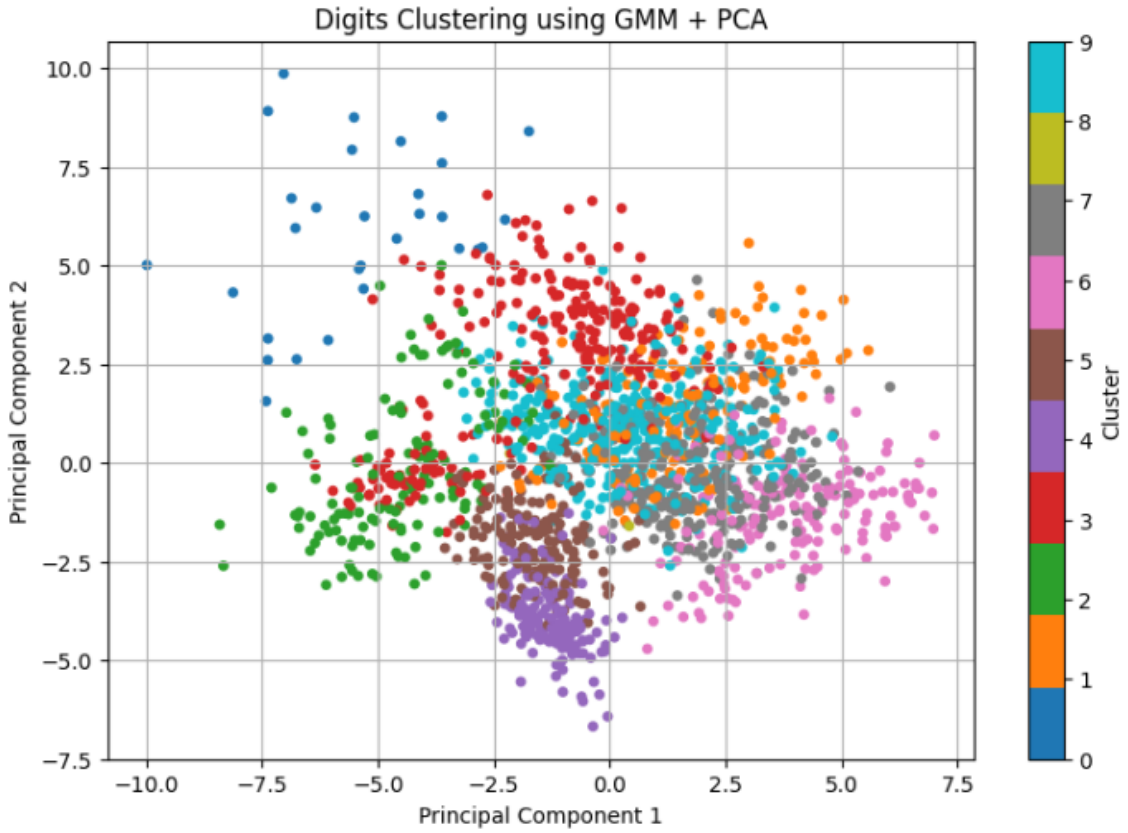
```python

# ------------------------------------------------------------
# STEP 9: Visualize K-Means Clusters
# ------------------------------------------------------------

plt.figure(figsize=(9, 6))

scatter = plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=kmeans_labels,
    cmap="tab10",
    s=15
)

plt.title("Digits Clustering using K-Means + PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(scatter, label="Cluster")
plt.grid(True)
plt.show()

# ------------------------------------------------------------
# STEP 10: Visualize GMM Clusters
# ------------------------------------------------------------

plt.figure(figsize=(9, 6))

scatter = plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=gmm_labels,
    cmap="tab10",
    s=15
)

plt.title("Digits Clustering using GMM + PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(scatter, label="Cluster")
plt.grid(True)
plt.show()

# ------------------------------------------------------------
# STEP 11: Display GMM Cluster Probabilities
# ------------------------------------------------------------

probabilities = gmm.predict_proba(X_scaled)

print("\nGMM Cluster Probabilities for First 5 Samples:")

print(
    pd.DataFrame(
        probabilities[:5],
        columns=[f"Cluster_{i}" for i in range(n_clusters)]
    ).round(3)
)

# ------------------------------------------------------------
# FINAL RESULT
# ------------------------------------------------------------

print("\n====================================")
print("EXPERIMENT 2 COMPLETED")
print("====================================")

print("K-Means Silhouette Score:",
      round(kmeans_silhouette, 4))

print("GMM Silhouette Score:",
      round(gmm_silhouette, 4))

print("K-Means ARI:",
      round(kmeans_ari, 4))

print("GMM ARI:",
      round(gmm_ari, 4))
```

Output:

```
Dataset Shape:
(1797, 64)

Number of Classes:
10

First 5 target values:
[0 1 2 3 4]

Standardized data shape:
(1797, 64)

================================================
CLUSTERING PERFORMANCE COMPARISON
================================================
                Algorithm  Silhouette Score  Adjusted Rand Index  Normalized Mutual Information
                  K-Means          0.139377             0.534407                       0.671244
Gaussian Mixture Model          0.117905             0.546745                       0.690155

PCA Explained Variance Ratio:
[0.12033916 0.09561054]
Total Variance Explained: 21.59 %
```



...




```
GMM Cluster Probabilities for First 5 Samples:
   Cluster_0  Cluster_1  Cluster_2  Cluster_3  Cluster_4  Cluster_5  \
0        0.0        0.0        0.0        0.0        0.0        1.0
1        0.0        0.0        0.0        0.0        0.0        0.0
2        0.0        0.0        0.0        0.0        0.0        0.0
3        0.0        0.0        0.0        0.0        0.0        0.0
4        0.0        0.0        1.0        0.0        0.0        0.0

   Cluster_6  Cluster_7  Cluster_8  Cluster_9
0        0.0        0.0        0.0        0.0
1        0.0        0.0        0.0        1.0
2        0.0        0.0        0.0        1.0
3        0.0        1.0        0.0        0.0
4        0.0        0.0        0.0        0.0

======================================
EXPERIMENT 2 COMPLETED
======================================
K-Means Silhouette Score: 0.1394
GMM Silhouette Score: 0.1179
K-Means ARI: 0.5344
GMM ARI: 0.5467
```

Result:

Thus, the Gaussian Mixture Model using the EM algorithm was successfully applied to the Digits dataset and its clustering performance was compared with K-Means using clustering evaluation metrics. PCA was used to visualize the clusters in two dimensions.

3) Dimensionality Reduction using PCA, Factor Analysis, and ICA:Load the Wine dataset from Scikit-learn and standardize the features. Apply Principal Component Analysis (PCA), Factor Analysis (FA), and Independent Component Analysis (ICA) to reduce the dimensionality and compare their outputs. Discuss how these techniques help address the Curse of Dimensionality through visualizations and component analysis.

Answer)

Definitions:

Dimensionality Reduction:

Dimensionality reduction is the process of reducing the number of features in a dataset while retaining the most important information.

For example:

13 features → 2 components

This makes data easier to visualize and can reduce computational complexity.

Curse of Dimensionality:

The Curse of Dimensionality refers to problems that occur when the number of features becomes very large.

High-dimensional data can lead to:

- Increased computational cost
- More complex models
- Difficulty in visualization
- Increased noise
- Poor performance in some machine learning algorithms

PCA:

Principal Component Analysis transforms the original features into a set of uncorrelated principal components.

The first component captures the maximum possible variance.

Factor Analysis:

Factor Analysis attempts to explain observed variables using a smaller number of hidden or latent factors.

It is commonly used when the observed variables are believed to be influenced by underlying factors.

ICA:

Independent Component Analysis transforms the original data into components that are as statistically independent as possible.

ICA is useful when the observed data is assumed to be a mixture of independent underlying sources.

PCA vs FA vs ICA:

Technique	**Main Idea**
PCA	Maximum variance
Factor Analysis	Find latent factors
ICA	Find statistically independent components

Objective:

To reduce the dimensionality of the Wine dataset using PCA, Factor Analysis and ICA, compare the resulting components, visualize the reduced data, and understand how dimensionality reduction helps address the Curse of Dimensionality.

Python Code:

```python
# ============================================================
# EXPERIMENT 3
# Dimensionality Reduction using PCA, Factor Analysis and ICA
# ============================================================

# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import (
    PCA,
    FactorAnalysis,
    FastICA
)

# ------------------------------------------------------------
# STEP 1: Load Wine Dataset
# ------------------------------------------------------------

wine = load_wine()

X = wine.data
y = wine.target

feature_names = wine.feature_names

print("Dataset Shape:")
print(X.shape)

print("\nFeature Names:")
print(feature_names)

print("\nFirst 5 Target Values:")
print(y[:5])

# ------------------------------------------------------------
# STEP 2: Create DataFrame
# ------------------------------------------------------------

df = pd.DataFrame(
    X,
    columns=feature_names
)

print("\nFirst 5 rows:")
print(df.head())

# ------------------------------------------------------------
# STEP 3: Standardize Features
# ------------------------------------------------------------

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\nStandardized data:")
print(X_scaled[:5])

# ------------------------------------------------------------
# STEP 4: PCA
# ------------------------------------------------------------

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

print("\n====================================")
print("PCA RESULTS")
print("====================================")

print("PCA Component Shape:")
print(X_pca.shape)

print("\nPCA Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

print(
    "\nTotal Variance Explained by PCA:",
    round(sum(pca.explained_variance_ratio_) * 100, 2),
```

```
    "%"
)

# ------------------------------------------------------------
# STEP 5: Factor Analysis
# ------------------------------------------------------------

fa = FactorAnalysis(
    n_components=2,
    random_state=42
)

X_fa = fa.fit_transform(X_scaled)

print("\n====================================")
print("FACTOR ANALYSIS RESULTS")
print("====================================")

print("Factor Analysis Component Shape:")
print(X_fa.shape)

# Calculate approximate variance representation
fa_variance = np.var(X_fa, axis=0)

print("\nVariance of Factor Components:")
print(fa_variance)

# ------------------------------------------------------------
# STEP 6: Independent Component Analysis
# ------------------------------------------------------------

ica = FastICA(
    n_components=2,
    random_state=42,
    max_iter=2000,
    whiten="unit-variance"
)

X_ica = ica.fit_transform(X_scaled)

print("\n====================================")
print("ICA RESULTS")
print("====================================")

print("ICA Component Shape:")
print(X_ica.shape)

print("\nICA Component Variance:")
print(np.var(X_ica, axis=0))

# ------------------------------------------------------------
# STEP 7: Create Comparison DataFrame
# ------------------------------------------------------------

comparison = pd.DataFrame({
    "Method": [
        "PCA",
        "Factor Analysis",
        "ICA"
    ],

    "Component 1 Variance": [
        np.var(X_pca[:, 0]),
        np.var(X_fa[:, 0]),
        np.var(X_ica[:, 0])
    ],

    "Component 2 Variance": [
        np.var(X_pca[:, 1]),
        np.var(X_fa[:, 1]),
        np.var(X_ica[:, 1])
    ]
})

print("\n====================================")
print("DIMENSIONALITY REDUCTION COMPARISON")
print("====================================")

print(comparison.to_string(index=False))

# ------------------------------------------------------------
```

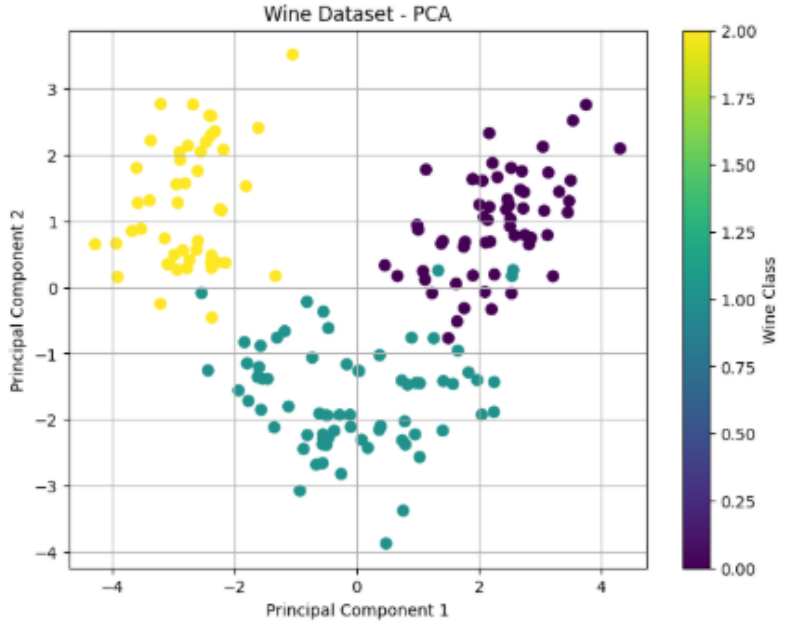
```python
# STEP 8: PCA Visualization
# ------------------------------------------------------------

plt.figure(figsize=(8, 6))

scatter = plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y,
    cmap="viridis",
    s=45
)

plt.title("Wine Dataset - PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(scatter, label="Wine Class")
plt.grid(True)
plt.show()

# ------------------------------------------------------------
# STEP 9: Factor Analysis Visualization
# ------------------------------------------------------------

plt.figure(figsize=(8, 6))

scatter = plt.scatter(
    X_fa[:, 0],
    X_fa[:, 1],
    c=y,
    cmap="viridis",
    s=45
)

plt.title("Wine Dataset - Factor Analysis")
plt.xlabel("Factor 1")
plt.ylabel("Factor 2")
plt.colorbar(scatter, label="Wine Class")
plt.grid(True)
plt.show()

# ------------------------------------------------------------
# STEP 10: ICA Visualization
# ------------------------------------------------------------

plt.figure(figsize=(8, 6))

scatter = plt.scatter(
    X_ica[:, 0],
    X_ica[:, 1],
    c=y,
    cmap="viridis",
    s=45
)

plt.title("Wine Dataset - Independent Component Analysis")
plt.xlabel("Independent Component 1")
plt.ylabel("Independent Component 2")
plt.colorbar(scatter, label="Wine Class")
plt.grid(True)
plt.show()

# ------------------------------------------------------------
# STEP 11: Compare Original and Reduced Dimensions
# ------------------------------------------------------------

print("\n====================================")
print("DIMENSION REDUCTION")
print("====================================")

print("Original number of features:", X.shape[1])

print("PCA features:", X_pca.shape[1])

print("Factor Analysis features:", X_fa.shape[1])

print("ICA features:", X_ica.shape[1])

print("\nDimensionality reduction:")

print(
    f"PCA: {X.shape[1]} -> {X_pca.shape[1]}"
```

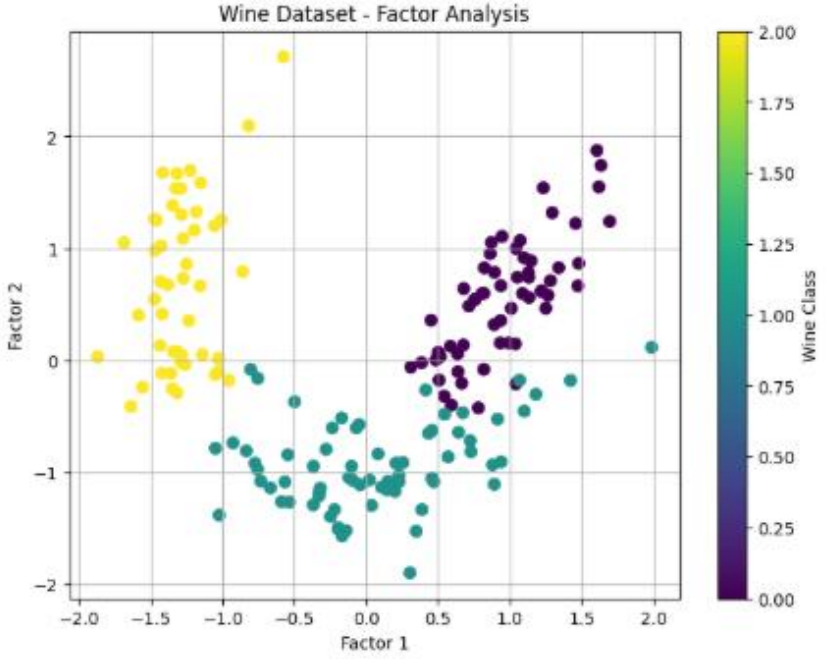
```
)

print(
    f"Factor Analysis: {X.shape[1]} -> {X_fa.shape[1]}"
)

print(
    f"ICA: {X.shape[1]} -> {X_ica.shape[1]}"
)

# ------------------------------------------------------------
# FINAL RESULT
# ------------------------------------------------------------

print("\n====================================")
print("EXPERIMENT 3 COMPLETED")
print("====================================")

print(
    "PCA retained",
    round(sum(pca.explained_variance_ratio_) * 100, 2),
    "% of the variance using 2 components."
)

print(
    "All three techniques successfully reduced",
    X.shape[1],
    "features to 2 components."
)
```

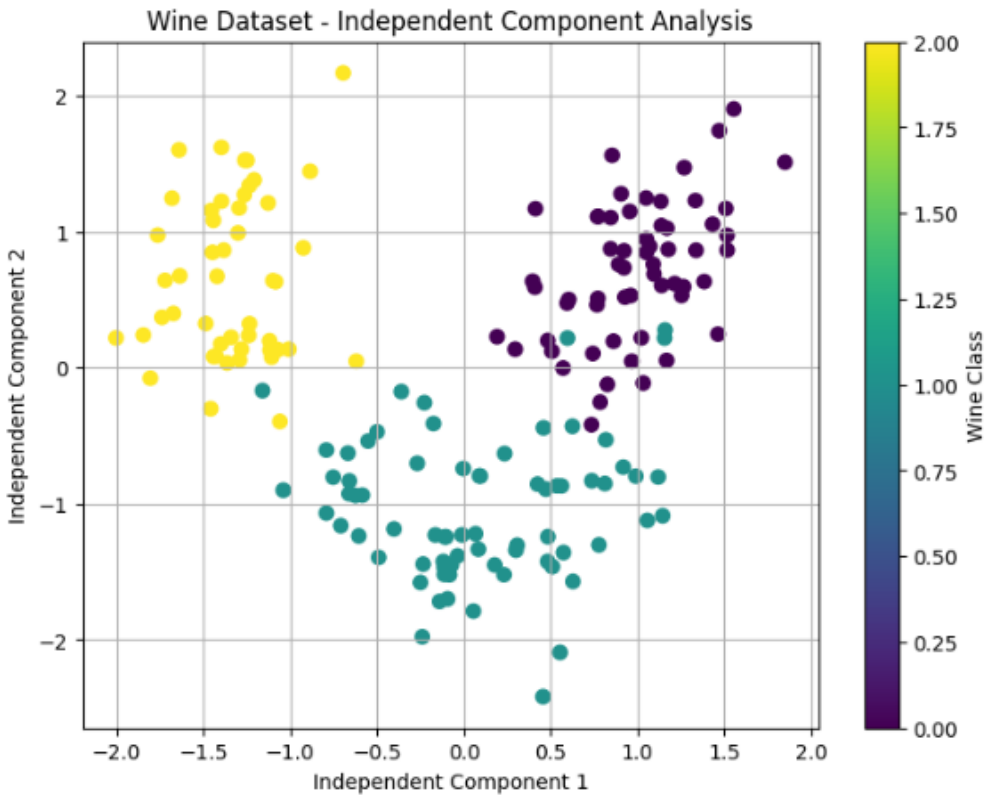
Output:

```
Dataset Shape:
(178, 13)

Feature Names:
['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity', 'hue', 'od280/od315_of_diluted_wines', 'proline']

First 5 Target Values:
[0 0 0 0 0]

First 5 rows:
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23        1.71  2.43               15.6      127.0           2.80
1    13.20        1.78  2.14               11.2      100.0           2.65
2    13.16        2.36  2.67               18.6      101.0           2.80
3    14.37        1.95  2.50               16.8      113.0           3.85
4    13.24        2.59  2.87               21.0      118.0           2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
0        3.06                  0.28             2.29             5.64  1.04
1        2.76                  0.26             1.28             4.38  1.05
2        3.24                  0.30             2.81             5.68  1.03
3        3.49                  0.24             2.18             7.80  0.86
4        2.69                  0.39             1.82             4.32  1.04

   od280/od315_of_diluted_wines  proline
0                          3.92   1065.0
1                          3.40   1050.0
2                          3.17   1185.0
3                          3.45   1480.0
4                          2.93    735.0

Standardized data:
[[ 1.51861254 -0.5622498   0.23205254 -1.16959318  1.91390522  0.80899739
   1.03481896 -0.65956311  1.22488398  0.25171685  0.36217728  1.84791957
   1.01300893]
 [ 0.24628963 -0.49941338 -0.82799632 -2.49084714  0.01814502  0.56864766
   0.73362894 -0.82071924 -0.54472099 -0.29332133  0.40605066  1.1134493
   0.96524152]
 [ 0.19687903  0.02123125  1.10933436 -0.2687382   0.08835836  0.80899739
   1.21553297 -0.49840699  2.13596773  0.26901965  0.31830389  0.78858745
   1.39514818]
 [ 1.69154964 -0.34681064  0.4879264  -0.80925118  0.93091845  2.49144552
   1.46652465 -0.98187536  1.03215473  1.18606801 -0.42754369  1.18407144
   2.33457383]
 [ 0.29570023  0.22769377  1.84040254  0.45194578  1.28198515  0.80899739
   0.66335127  0.22679555  0.40140444 -0.31927553  0.36217728  0.44960118
  -0.03787401]]

====================================
PCA RESULTS
====================================
PCA Component Shape:
(178, 2)

PCA Explained Variance Ratio:
[0.36198848 0.1920749 ]

Total Variance Explained by PCA: 55.41 %
```

```
...
==================================
FACTOR ANALYSIS RESULTS
==================================
Factor Analysis Component Shape:
(178, 2)

Variance of Factor Components:
[0.95605362 0.87119034]

==================================
ICA RESULTS
==================================
ICA Component Shape:
(178, 2)

ICA Component Variance:
[1. 1.]

==================================
DIMENSIONALITY REDUCTION COMPARISON
==================================
          Method  Component 1 Variance  Component 2 Variance
             PCA              4.705850              2.496974
Factor Analysis              0.956054              0.871190
             ICA              1.000000              1.000000
```








```
====================================
DIMENSION REDUCTION
====================================
Original number of features: 13
PCA features: 2
Factor Analysis features: 2
ICA features: 2

Dimensionality reduction:
PCA: 13 -> 2
Factor Analysis: 13 -> 2
ICA: 13 -> 2

====================================
EXPERIMENT 3 COMPLETED
====================================
PCA retained 55.41 % of the variance using 2 components.
All three techniques successfully reduced 13 features to 2 components.
```

Result:

Thus, the Wine dataset was successfully reduced from 13 dimensions to 2 dimensions using PCA, Factor Analysis and ICA. PCA focuses on preserving maximum variance, Factor Analysis identifies latent factors, and ICA attempts to identify statistically independent components. These dimensionality reduction techniques help make high-dimensional data easier to visualize and analyze and help address issues associated with the Curse of Dimensionality.